

THE **AI** CODEBASE ANALYST

FROM ZERO TO A WORKING AGENT

LESSON 2

THE AGENT AS A STATE MACHINE

Lesson 2: The Agent as a State Machine

"Simple things should be simple, complex things should be possible."

— Alan Kay

The "Why": The Amnesiac Tool

In Lesson 1, we built a secure, observable on-ramp to the world of LLMs. We can call a model, get a response, and see the trace. But our script has a fatal flaw: **it has amnesia**.

It takes an input, produces an output, and stops. It cannot hold a conversation, remember previous messages, or reason in steps. It is a one-shot tool, not an agent. Our "skilled intern" can't learn if they forget everything the moment a task is done.

To build a true agent, we need to solve three core problems:

1. **Memory:** How does it remember the conversation history to maintain context?
2. **Reasoning Flow:** How can it loop, make decisions, or perform multi-step tasks?
3. **Clarity:** As logic gets more complex, how do we prevent our code from becoming an unmanageable mess of "if/else" statements—a black box of our own making?

A simple function call cannot do these things. We need a new architecture.

The "What": The Graph as a Brain

The solution is to model our agent not as a script, but as a **State Machine**. Think of a flowchart: you start at a specific point, perform an action in a "box," and follow an "arrow" to the next box. This structure provides the memory and reasoning flow we need.

We will use **LangGraph** to build this state machine explicitly. LangGraph is the modern, official library for creating robust and observable agents. It forces us to think like architects by defining three core components:

1. **State:** A Python object that represents the agent's memory at any given moment. We are making memory an *explicit*, first-class citizen of our system.

2. **Nodes:** Python functions that represent the "boxes" in our flowchart. Each node is a "worker" that performs an action, like calling an LLM, and modifies the agent's state.
3. **Edges:** The "arrows" in our flowchart. They define the pathways between nodes, directing the flow of logic.

By building our agent this way, we create a system that is transparent, easy to debug, and powerful enough to handle complex, multi-step tasks. We are building the scaffolding of accountability for our "Liar, Slacker, and Thief."

The "How" (The Build): Step-by-Step

We will build upon our project from Lesson 1.

1. Activate Your Virtual Environment

If it's not already active, navigate to your codebase-analyst directory and run:

```
# For macOS/Linux
source venv/bin/activate
```

```
# For Windows
.\venv\Scripts\activate
```

2. Install New Dependencies

We need to add the langgraph library to our project.

```
pip install langgraph
```

3. Update requirements.txt (Good Practice)

It's good practice to keep your requirements file up-to-date.

```
pip freeze > requirements.txt
```

Your project directory is now set up and ready for the new code.

Code Walkthrough: Building the Agent's Brain

This is where we build the agent's core reasoning loop. We will create a new file that defines and runs our first conversational agent.

- **Filename:** 02_agent_state_machine.py

```
# 02_agent_state_machine.py

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

# --- 1. Load Environment Variables ---
load_dotenv()

# --- 2. Define the Agent's State (Its Memory) ---
class AgentState(TypedDict):
    messages: List[HumanMessage | AIMessage]

# --- 3. Define the Nodes (The "Workers") ---
def call_model(state: AgentState):
```

```
"""A node that calls the LLM to generate a response."""
print("---CALLING THE MODEL---")
llm = ChatOpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY"),
    model="google/gemini-2.5-flash",
)
response = llm.invoke(state["messages"])
return {"messages": [response]}

# --- 4. Define the Graph (The Agent's "Brain" Structure) ---
graph_builder = StateGraph(AgentState)
graph_builder.add_node("call_model", call_model)
graph_builder.set_entry_point("call_model")
graph_builder.add_edge("call_model", END)

# --- 5. Compile the Graph into a Runnable ---
agent_graph = graph_builder.compile()

# --- 6. Run the Agent in a Conversational Loop ---
if __name__ == "__main__":
    print("Hello! I am your AI assistant. How can I help you today?")
    print("Type 'exit' to end the conversation.")

    messages = []
    while True:
        user_input = input("You: ")
        if user_input.lower() == "exit":
            break

        messages.append(HumanMessage(content=user_input))
        response = agent_graph.invoke({"messages": messages})
        messages = response["messages"]
        ai_message = messages[-1].content
        print(f"AI: {ai_message}")
```

Deconstructing the Script

- **Section 2: The Agent's State:** This is the most important new concept. AgentState is our agent's memory. By using TypedDict, we get auto-completion and type-checking from our editor, a professional practice that prevents bugs. For now, our agent's entire memory is just a list of messages.
- **Section 3: The call_model Node:** This is our first "worker." It's a Python function that takes the current state as an argument, calls our chosen Gemini model, and returns a dictionary. LangGraph uses this dictionary to update the state, appending the new AIMessage to the list.
- **Section 4: The Graph Definition:** Here, we act as architects.
 - StateGraph(AgentState): We create a blueprint for a graph that will manage our specific AgentState.
 - .add_node("call_model", call_model): We register our worker function, giving it a unique string name.
 - .set_entry_point("call_model"): We tell the graph, "When you start, run this node first."
 - .add_edge("call_model", END): We define the logic flow: "After the 'call_model' node finishes, the process is complete (END)."
- **Section 6: The Interactive Loop:** The while loop creates a simple chat interface. The key to making it conversational is that the messages list exists *outside* the loop. We add the user's new message, invoke the graph, and then update our local messages list with the graph's final state, which now includes the AI's response. This ensures the full conversation history is sent to the model on the next turn.

The Integrated Run: A Conversation

Prerequisites:

- An active (venv) virtual environment with langgraph installed.
- A valid OPENROUTER_API_KEY in your .env file.

Run Command:

From your terminal, execute the Python script:

```
python 02_agent_state_machine.py
```

Expected Console Output:

Your conversation will be interactive, and the AI will remember what you said previously.

```
Hello! I am your AI assistant. How can I help you today?
```

```
Type 'exit' to end the conversation.
```

```
You: I'm learning about AI agents. What's the core idea behind  
LangGraph?
```

```
---CALLING THE MODEL---
```

```
AI: The core idea behind LangGraph is to build AI agents as state  
machines. Instead of a single, complex prompt, you define the agent's  
logic as a graph of nodes (actions) and edges (transitions). This makes  
the agent's reasoning process explicit, transparent, and much easier to  
debug and control, especially for complex, multi-step tasks.
```

```
You: why is that better than the old way?
```

```
---CALLING THE MODEL---
```

```
AI: It's better than older methods for a few key reasons. By making the  
reasoning explicit in a graph, you get enhanced reliability because you  
can control error handling at each step. It's also far more observable;  
you can trace the agent's path through the graph, which is invaluable  
for debugging. Finally, this modular structure allows for much greater  
complexity, including cycles for retries and conditional logic that was  
difficult to manage in a single prompt.
```

```
You: exit
```

Analysis: The System-Level View

We have created our first true agentic loop. Each time you send a message, the graph executes its simple logic. Go check your LangSmith traces—you will see each turn of the conversation logged as a separate run. Inspect the trace to see the full list of messages sent to the model in each turn, confirming that the agent's memory is growing.

Under the Hood: Why LangGraph is the Modern Standard

You might see older tutorials that use a class called `AgentExecutor`. This was the original method for building agents in LangChain, but the official documentation now considers it a legacy pattern. The reason for the shift is **control and transparency**. The legacy `AgentExecutor` worked like a "magic box," hiding its internal logic. LangGraph, in contrast, forces us to build an explicit flowchart (a graph) of our agent's logic. This makes our agent transparent, easier to debug, and simpler to customize—all hallmarks of a production-grade system. By learning LangGraph, you are learning the modern, official, and future-proof standard.

Summary & Key Takeaways

This lesson was a monumental step. You have moved from writing simple scripts to architecting an agent's brain. This is the foundational pattern for the entire course.

Mental Models You've Gained:

- **Agents are State Machines:** You now think of agents not as a single prompt, but as a graph of states and actions. This is the key to managing complexity and building resilient systems.
- **Explicit State is Power:** You understand that an agent's memory (`AgentState`) is a first-class citizen that you define and control, not a hidden side effect.
- **Nodes are Workers, Edges are Logic:** You can separate the *actions* (nodes) from the *flow of control* (edges), which is the key to building complex, maintainable agents.
- **Compilation Creates Runnables:** You know that defining a graph is a blueprint; compiling it creates a concrete object you can execute.

What's Next

Our agent can now hold a conversation. It has a brain and a memory. **But it has no hands.** It lives entirely within the confines of its chat with the LLM and has no ability to interact with the outside world. It can't read files, browse the web, or run code.

In Lesson 3, we will solve this problem by giving our agent its first **Tool**. We will teach it how to perform an action, transforming it from a simple conversationalist into a true analyst that can interact with its environment.